

路口车道标注功能详细技术方案

覆盖无人机悬停识别、自动生成标注任务、人工车道标注、参数持久化与后续检测复用

项目	内容
产品定位	无人机路口交通态势监测平台，面向巡飞视频检测、交通流统计、告警和 GIS 指挥。
本方案范围	标定中心 -> 车道标注；监控流程检测器；Kafka/Platform/Console 联动。
生成日期	2026-06-20
当前验证数据	inter_xqh 视频 + SRT 遥测，标注任务 lane-INT_camera_3-1781271962。

方案结论

系统采用“检测管道发现悬停 -> Kafka 发送快照与遥测 -> 平台生成标注任务 -> Console 人工绘制车道 -> 导出 roads/lanes JSON -> 后续 PipelineManager 自动复用”的闭环。该路径保持检测流、平台管理和前端标定页面解耦，并允许离线视频回放与真实无人机 RTSP 任务使用同一套契约。

1. 产品与建设目标

TrafficAnalyzer 是无人机路口交通态势监测平台，核心管道从 MP4、RTSP 或摄像头读取视频，完成 YOLO 检测、ByteTrack 跟踪、无人机运动补偿、道路/车道统计，并通过 Kafka、WebSocket 和 Console 页面形成实时监控闭环。本方案聚焦巡飞转悬停后的车道标注能力。

业务目标是减少每次任务启动前手工准备车道参数的成本。当无人机在某个路口稳定悬停超过阈值后，系统自动截取检测画面生成待标注任务；人工在标定中心完成车道多边形绘制并保存；后续同一路口再次启动检测流时，平台优先使用已保存的标注参数。

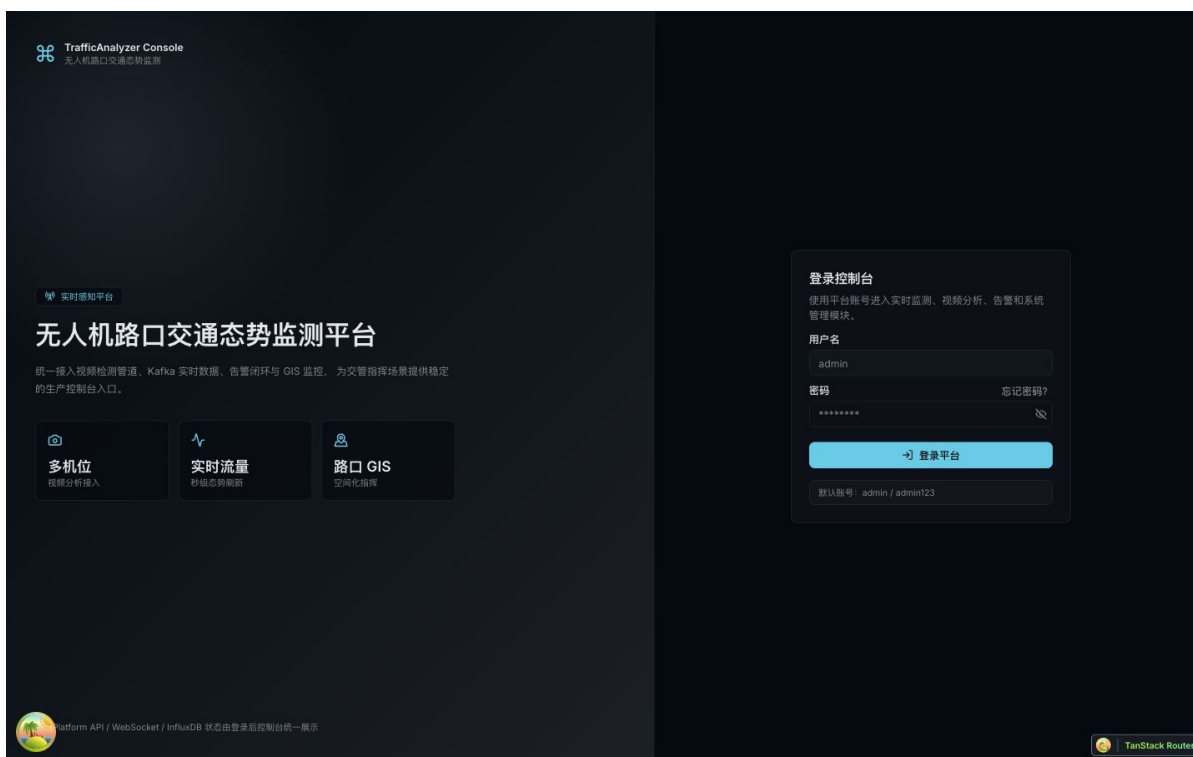


图 1：Console 登录入口，统一进入实时监控、视频分析、告警和标定模块。

2. 功能范围

能力	说明	验收要点
悬停识别	检测管道根据 MotionCompensationNode 输出的 is_hovering 与 drone_position 判断稳定悬停。	同一路口固定坐标超过 30 秒，且半径误差不超过配置值。
任务生成	KafkaProducerNode 在 stats 消息中带 road_polygons、drone_position、is_hovering 与 annotation_snapshot_jpeg。	平台 Consumer 收到稳定悬停消息后创建 lane task，包含图片 URL 和尺寸。
人工标注	标定中心的车道标注页从任务列表读取图片，SVG 画布点击顶点、闭合多边形并保存。	保存 payload 包含 lane_id、direction、polygon、roads。
参数复用	LaneAnnotationStore 导出口路级 JSON，PipelineManager 在启动检测流时解析并优先使用。	后续同 intersection_id 未显式指定 roads_json 时自动使用 export_path。

3. 页面截图与操作流

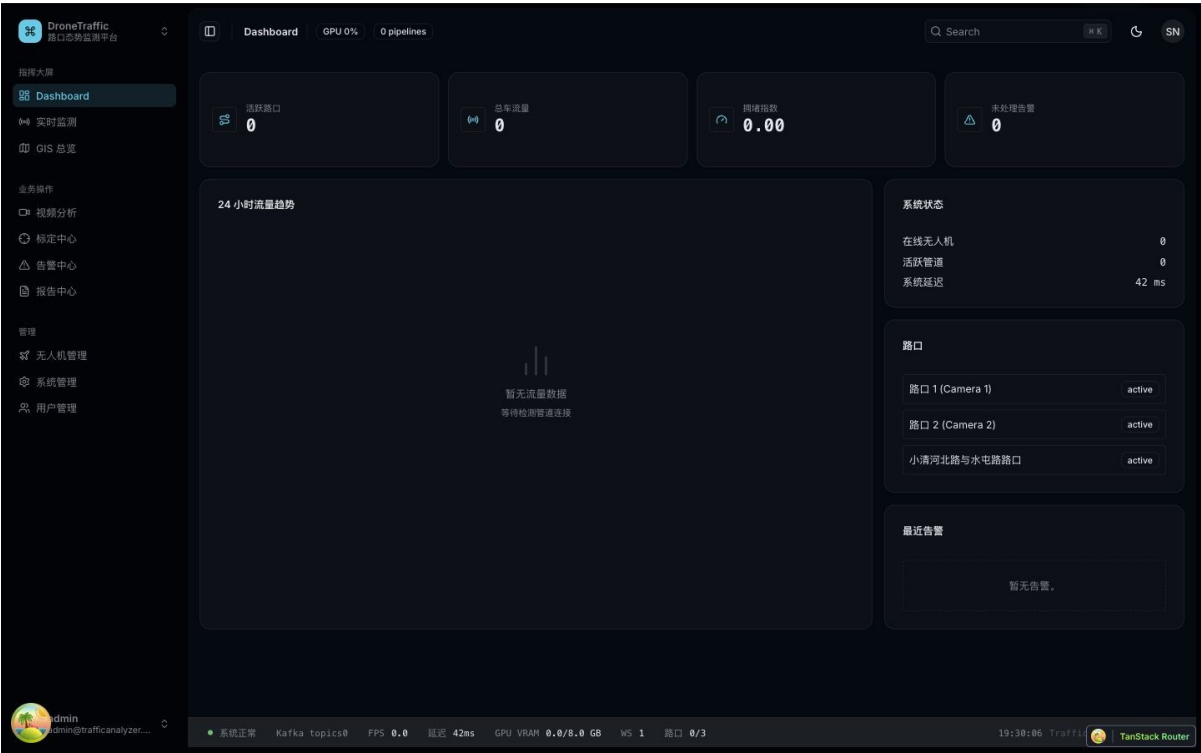


图 2：指挥大屏，展示活跃路口、总车流量、拥堵指数和系统状态。

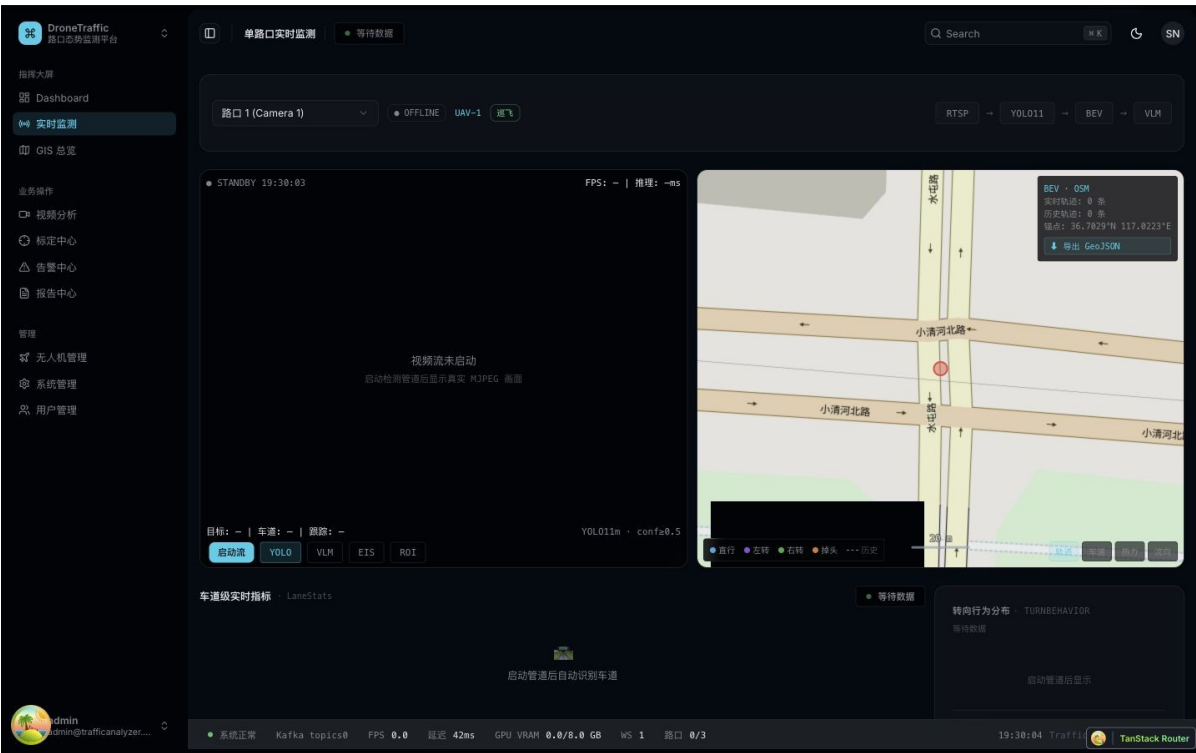


图 3：实时监测页面，操作员从路口/无人机选择检测流，并观察视频与 BEV 地图。

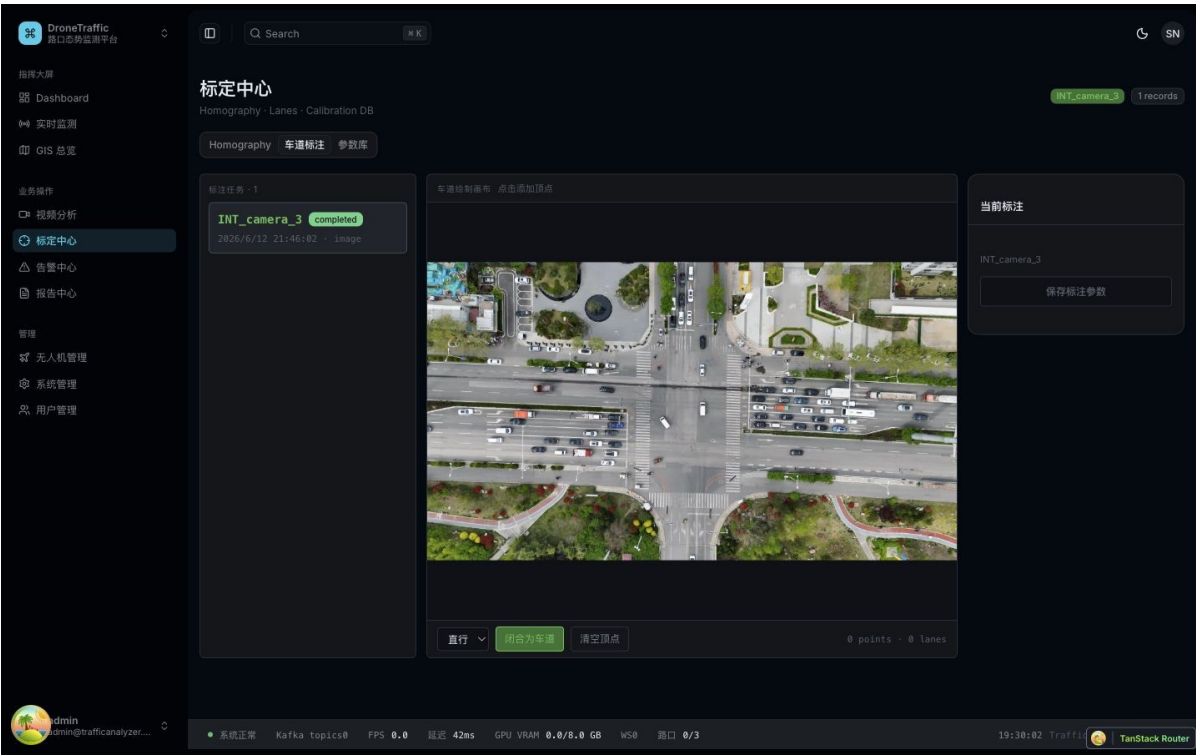


图 4：标定中心车道标注页，左侧为悬停生成的任务，中间为真实无人机快照画布。

4. 总体架构

车道标注闭环跨越检测管道、Kafka 消息、平台服务和 React Console 四层。

无人机视频/离线视频

-> main_optimized.py 三进程检测管道

-> MotionCompensationNode 输出 drone_position / is_hovering

-> KafkaProducerNode 输出 statistics_N + annotation_snapshot_jpeg

-> Platform KafkaConsumerService.observe_stats()

-> LaneAnnotationStore 创建 lane task 与 JPEG 图片

-> Console 标定中心拉取任务并绘制车道

-> Calibration API 保存 lanes/roads

-> 导出 lane_annotations/{intersection_id}.json

-> PipelineManager 后续启动同一路口检测流时自动复用

层级	组件	责任
检测管道	main_optimized.py / KafkaProducerNode	计算交通指标、发送 stats、编码用于标注的 JPEG 快照。
消息层	Kafka statistics_N	把无人机位置、悬停状态、路口多边形和图片快照传递到平台。
平台层	KafkaConsumerService / LaneAnnotationStore / Calibration API	识别稳定悬停、生成任务、保存标注、暴露图片与参数接口。
前端层	traffic-fly-console 标定中心	任务列表、车道绘制画布、方向选择、保存标注参数。
复用层	PipelineManager	启动检测流时解析同一路口保存的 export_path。

5. 核心流程设计

5.1 监控流程启动

操作员在实时监测页面选择路口后点击“启动流”。前端通过 Pipeline API 传入 drone_id、intersection_id、video_src、telemetry_source 和 telemetry_file_path。平台创建子进程运行 main_optimized.py，并通过环境变量注入 VIDEO_SRC、ROADS_JSON、TOPIC_NAME、CAMERA_ID、INTERSECTION_ID 和 VIDEO_PORT。

5.2 悬停检测与快照生产

MotionCompensationNode 基于遥测产生无人机 ENU 位置、速度和 is_hovering。KafkaProducerNode 周期性发送 stats，若当前帧存在 frame_result 或 frame，则按配置宽度压缩成 JPEG，并以 base64 字段 annotation_snapshot_jpeg 附在消息内。

5.3 平台生成标注任务

KafkaConsumerService 在 _handle_stats 中调用 LaneAnnotationStore.observe_stats。Store 按 intersection_id 维护悬停状态：非悬停清空状态；坐标变化超过 hover_radius_m 重新计时；稳定时间超过 hover_seconds 且该路口尚无 active annotation 时，创建 pending task，并把 JPEG 解码保存到 lane_task_images。

5.4 人工标注与保存

Console 标定中心通过 GET /api/v1/calibration/lane-tasks 拉取任务。用户在 SVG 画布上点击顶点，至少 3 个点后闭合为车道，选择方向 straight、left_turn、right_turn 或 u_turn。保存时提交 lanes 和 roads，平台规范化 polygon 数值并写入 DB。

5.5 后续检测复用

保存成功后，Store 导出 lane_annotations/{intersection_id}.json。Pipeline API 收到新的启动请求时，如果调用方未显式指定 roads_json，则根据 intersection_id 查找已保存 annotation，并把 export_path 作为实际 roads_json。检测管道中的 VideoReader 和 LaneDetectionNode 后续可直接加载 manual 标注。

6. 数据契约

消息字段	类型	说明
intersection_id	string	统一路口 ID，例如 INT_camera_3。
drone_position	object	包含 anchor_lat、anchor_lon、easting_m、northing_m。
is_hovering	boolean	检测管道判断是否悬停。
road_polygons	object	当前检测配置中的道路多边形，用于保存标注时继承 roads。
annotation_snapshot_jpeg	string	base64 JPEG；平台创建任务时解码保存为图片。
annotation_snapshot_width/height	number	前端 SVG viewBox 和图片尺寸。

REST API	方法	用途
/api/v1/calibration/lane-tasks	GET	列出悬停生成的车道标注任务。
/api/v1/calibration/lane-tasks/{task_id}/image	GET	返回任务快照 JPEG；图片地址可被 SVG image 直接加载。
/api/v1/calibration/lane-tasks/{task_id}/annotation	POST	保存人工标注 lanes 与 roads。
/api/v1/calibration/lane-annotations	GET	列出已保存可复用的路口车道参数。
/api/v1/pipelines	POST	启动检测流；默认使用同路口已保存的标注参数。

7. 存与文件构

```
output/inter_xqh_page_e2e/  
  lane_annotation_db.json  
  lane_task_images/  
    lane-INT_camera_3-1781271962.jpg  
  lane_annotations/  
    INT_camera_3.json
```

lane_annotation_db.json 是轻量任务库，包含 tasks 与 annotations 两个集合。图片按 task_id 命名，导出参数按 intersection_id 命名，便于后续 PipelineManager 无状态读取。

8. 前端交互设计

车道标注页采用三栏布局：左栏任务列表，中间 SVG 绘制画布，右栏当前标注摘要和保存按钮。画布以任务图片尺寸作为 viewBox，点击坐标直接落到原图坐标系，避免额外换算误差。

交互	规则
选择任务	默认选择最新任务，也可点击左侧任务卡切换。
添加顶点	点击画布添加点，点位以整数像素坐标记录。
闭合车道	至少 3 个顶点才能闭合为 lane polygon。
方向选择	下拉选择直行、左转、右转、掉头。
保存	只有存在 selectedTask 且至少一个 draft lane 时可保存。

9. 运行配置

配置项	默认/示例	说明
LANE_ANNOTATION_DB_PATH	output/.../ lane_annotation_db.json	任务库路径，决定图片和导出目录。
LANE_ANNOTATION_HOVER_SECONDS	30	稳定悬停时长阈值。
LANE_ANNOTATION_HOVER_RADIUS_M	1.5	同一悬停点的最大偏移半径。
annotation_snapshot_width	960	KafkaProducerNode 快照压缩宽度。
annotation_snapshot_jpeg_quality	75	JPEG 压缩质量。

10. 安全与权限

平台 API 默认走 JWT 中间件。任务图片端点需要允许浏览器 SVG image 直接读取，因此 GET /lane-tasks/{task_id}/image 作为只读公共图片端点处理；保存标注的 POST 仍受 JWT 保护，避免未授权写入车道参数。

权限边界

公共化的仅是任务图片读取，不包括任务列表、annotation 保存或参数库查询。生产环境如需更强隔离，可将图片 URL 改为短期签名地址或由前端以 Blob URL 加载。

11. 验证方案

验证项	命令/方式	预期
后端语法	python -m py_compile 相关 Python 文件	无语法错误。
前端类型	pnpm -C traffic-fly-console exec tsc -b	TypeScript 编译通过。
任务图片	GET /api/v1/calibration/lane-tasks/{task_id}/image	返回 image/jpeg，尺寸与任务字段一致。
未授权写入	无 token POST annotation	返回 401。
标注复用	POST /api/v1/pipelines 不传 roads_json	返回 roads_json 为该路口导出 JSON 路径。
页面截图	登录、Dashboard、实时监测、标定中心	页面非空，标定页显示真实无人机快照。

12. 风险与演进

风险	影响	建议
不同高度/角度重复悬停	同一路口可能需要多套标注。	后续引入 altitude/gimbal_pitch 分桶或版本化 annotation。
图片公共读取	内部网络可直接访问快照。	生产环境改签名 URL 或鉴权后 Blob 加载。
文件型 DB 并发	多实例平台可能产生写冲突。	迁移到 PostgreSQL 表并使用唯一索引约束 pending task。
手绘精度	人工误点会影响车道统计。	加入撤销、拖拽顶点、已有车道编辑、缩放和平移。
道路数量历史硬编码	部分统计仍假定 road_1~road_5。	继续推进动态道路数组在 Telegraf/Grafana 的兼容改造。

13. 源码落点

文件	作用
nodes/KafkaProducerNode.py	发送 stats、路口多边形、悬停快照和无人机位置。
platform/app/kafka/consumer.py	消费 stats 并调用 LaneAnnotationStore.observe_stats。
platform/app/services/lane_annotation_store.py	任务生成、图片保存、annotation 持久化和导出。
platform/app/api/v1/calibration.py	车道标注任务、图片、保存和参数库 API。
platform/app/api/v1/pipelines.py	启动检测流时复用已保存 annotation export_path。
traffic-fly-console/src/features/calibration/index.tsx	车道标注页面和 SVG 绘制逻辑。
traffic-fly-console/src/features/monitoring/index.tsx	检测流启动入口。

14. 交付结论

该方案把无人机悬停检测、标注任务生产、人工车道绘制和检测参数复用串成闭环。它不改变既有视频检测主链路，只通过扩展 stats 消息和平台标定服务新增能力，因此对现有 Dashboard、GIS、告警链路影响较小。后续重点应放在标注编辑体验、annotation 版本化和生产级存储迁移。